

Raniwala ESSL Attendance Sync — Network Issue & Fix

Status: RESOLVED — stable in production. One follow-up change is planned (see Section 6).

1. Background — How ESSL Sync Normally Works

CrewCore's standard ESSL biometric integration (scripts/essl-sync-agent, migration 20260818_essl_integration.sql) is a **push** model:

```
ESSL machine -> eTimeTrackLite (local PC + MySQL/SQL Server)
               -> essl-sync-agent (Node.js, runs on that PC)
               -> essl-punch Edge Function
               -> attendance / punches tables
```

This requires a small agent process to run continuously on a Windows PC at the tenant's site, with:

- direct read access to the ESSL vendor software's local database (MySQL or SQL Server), and
- outbound HTTPS access from that PC to CrewCore's Supabase Edge Functions, to POST each new punch.

This is the model every other tenant with an ESSL machine uses today.

2. The Actual Issue at Raniwala

Raniwala's site network could not reliably support the standard agent model. It wasn't a single clean failure but a combination of site-level network restrictions:

- **Firewall restrictions** — outbound connections from the PC running eTimeTrackLite were blocked/restricted by site IT policy, so a persistent agent process making outbound HTTPS calls to Supabase couldn't be relied on to get through.
- **Domain/DNS issues** — the site's network/domain setup interfered with resolving or reaching CrewCore's endpoints consistently from that machine.
- **General connectivity** — on top of the above, the PC's internet connection itself was not dependable enough for a long-running sync process to stay up and not miss cycles.

In short: the constraint was at the **site's IT/network layer** (firewall + domain + connectivity), not a bug in the sync agent code itself. Running CrewCore's own outbound agent from inside Raniwala's network was not something their IT setup would support reliably.

3. What Was Tried

Only one real fix was attempted — the standard local-agent (push) model was ruled out early once the firewall/domain/connectivity picture became clear, and the team went straight to a different architecture rather than trying to patch around the network restrictions (VPN/tunnel/port-forwarding, static IP, retrying the agent, etc. were not pursued).

The fix: flip push → pull.

Raniwala separately already had a small internal “ESSL Web API” dashboard tool running on-site, which aggregates punches from all 3 of their on-site biometric terminals (via ESSL's own device API) and exposes them over HTTP at a fixed, internet-reachable ip:port (`http://183.83.176.221:8001/api/data`). That endpoint was reachable *from the outside* even though CrewCore's own agent couldn't reliably get *out* from inside Raniwala's network.

So instead of running a CrewCore agent on Raniwala's local network, CrewCore was changed to **poll that dashboard from the Supabase side**, sidestepping Raniwala's outbound firewall/domain problems entirely — the only outbound call now happens from Supabase's infrastructure, not from anything inside Raniwala's network.

4. How It Works Now (Final Architecture)

```
3x ESSL terminals -> Raniwala's own ESSL Web API dashboard (public ip:port)
                    |
                    | polled every 2 minutes by pg_cron
                    v
    essl-web-poll --> essl-punch --> attendance / punches
```

Pieces, in the order they were built:

1. `essl_web_poll_state` table

(`20260819_1_essl_web_poll_state.sql`) — one row per (`essl_employee_code`, `date`), storing the last known `last_in/last_out` seen for that employee/day. This is what lets each cycle tell “changed” rows apart from rows it's already forwarded.

2. `essl-web-poll` Edge Function

(`supabase/functions/essl-web-poll/index.ts`) — on each run:

- Fetches the full current dashboard payload (GET `.../api/data?days=1&force=1`, ~300 rows).
- Diffs it against `essl_web_poll_state` and keeps only rows whose in/out time actually changed since the last cycle.
- Forwards just those changed rows to the existing `essl-punch` Edge Function (the same one the local-agent model uses), using a per-tenant `RANIWALA_ESSL_API_KEY` — so status calculation (Present/Late/Half Day/Absent) is computed by the exact same code path either way, with zero duplicated business logic.
- Updates `essl_web_poll_state` with the new last-seen values, regardless of whether `essl-punch` accepted every row (an unmapped ESSL code today may get mapped to an employee later — the cycle shouldn't keep resending it forever either way).

3. `pg_cron` schedule

(`20260819_2_essl_web_poll_cron.sql`) — fires `essl-web-poll` every 2 minutes via `pg_net.http_post` (fire-and-forget from SQL's side). The request timeout was raised from `pg_net`'s 5s default to **25s**: the very first fire timed out purely on the Edge Function's cold start, and a cycle with several changed punches also needs more than 5s since `essl-punch` does a few DB round trips per punch. 25s comfortably fits inside the 2-minute interval between cycles.

4. A bug found and fixed during rollout — `WORKER_RESOURCE_LIMIT`

An earlier version of `essl-web-poll` re-sent the dashboard's *entire* ~300-row payload to `essl-punch` on every single 2-minute cycle. Because `essl-punch` does several DB round trips per punch, 300 punches every 2 minutes was far more work than the Edge Function's resource limit allowed, and the function started hitting `WORKER_RESOURCE_LIMIT`. The diff-against-state step (step 2 above) was added specifically to fix this — a normal cycle now only touches the handful of rows that actually changed since the last poll, not the whole dashboard.

5. Daily cleanup cron

(`20260819_3_essl_web_poll_cleanup.sql`) — `pg_net` permanently logs every request it makes in `net._http_response`. At one call every 2 minutes (~720/day) that grows forever if left alone, so a separate daily job (`17 3 * * *`, offset from the 2-minute mark so it never overlaps a sync fire) prunes anything older than 3 days — enough to debug a stuck cycle without keeping the table forever.

Punches synced this way are tagged source: `'device'` on the punches row, same as the local-agent model, so they're indistinguishable in the audit trail from any other ESSL device punch.

***Note for future tenants:** this is currently hardcoded to Raniwala only — `ESSL_WEB_API_URL` and `RANIWALA_ESSL_API_KEY` are literal values in `essl-web-poll/index.ts`, because Raniwala is the only tenant today with a web-dashboard ESSL source rather than direct SQL Server/MySQL access. If a second tenant needs this same pull-based path, that URL/key should move onto the `essl_devices` row (e.g. a `poll_url` column) and `essl-web-poll` should loop over every device configured that way, instead of being single-tenant.*

5. Current Status

Fully resolved and stable in production. The pull/poll model has been running reliably since it replaced the local-agent approach, with no recurring connectivity or resource-limit issues since the fix in Section 4.4 above.

6. Planned Follow-Up

The current source is Raniwala's own internal dashboard tool at a fixed public ip:port. The plan going forward is to have Raniwala push/upload their punch data to a **different, dedicated API** instead, with `essl-web-poll` (or its successor) fetching from that API rather than the current dashboard endpoint. This doc should be updated once that change lands — treat `ESSL_WEB_API_URL` in `essl-web-poll/index.ts` as provisional, not final.

Raniwala Office Network — CrewCore App Not Working

Status: *RESOLVED*.

This is a separate, unrelated incident from the ESSL sync issue documented above. That one was about attendance *punches* failing to sync from Raniwala's biometric machines. This one was about the **CrewCore application itself** — logging in, viewing data, essentially the whole app — not working *specifically when accessed from inside Raniwala's office network*, while working completely normally from anywhere else (home, mobile data, other offices).

What the Problem Actually Looked Like

Users on Raniwala's office Wi-Fi/LAN would open `crewcore.in` and the page itself would load (it's a static frontend, served from Vercel, and that domain was never blocked). But the moment the app tried to do anything that talks to the backend — logging in, fetching attendance, loading the dashboard, literally any Supabase call — it would fail. From the user's point of view this presented as: the app “hangs,” or login never completes, or every screen after login just shows blank/loading/error, **only on Raniwala's office network**. The exact same login, same device, same browser, worked fine the moment someone stepped outside that network (e.g. on mobile data) or from a different office. That “works everywhere except this one office” pattern is what pointed the investigation at Raniwala's own office network/internet setup, rather than at CrewCore's code or at Supabase itself.

Word-for-Word Explanation of the Actual Cause

CrewCore's frontend does not talk to `supabase.co` directly for its backend calls — it talks to a custom domain, `api.crewcore.in`, which is set up as a DNS **CNAME record pointing straight at CrewCore's Supabase project** (`yxueywgrqrfgynqknsqs.supabase.co`). Every single backend interaction the app makes — authentication/login, every database read/write through Supabase's REST layer, every Edge Function call (like `essl-punch`, `create-employee-user`, etc.), file storage, and the realtime/websocket connection used for live updates — all of it goes out to `api.crewcore.in`, which forwards straight through to Supabase.

The problem is in what that domain returns when something *other than* the CrewCore app hits it — specifically, when it's hit at the root path `/` with a plain, ordinary browser or automated request (exactly the kind of request a corporate network's own security/filtering appliance makes when it first sees a new domain, in order to decide what category to file it under). Because `api.crewcore.in` was a bare CNAME straight to Supabase with nothing else in front of it, that root-path request landed directly on Supabase's own gateway, which has no real page to serve there — it just returns Supabase's generic, contentless gateway response. There is no title, no description, no actual “this is what this site is” content of any kind for a filter to read.

Many corporate networks — and this is exactly what was happening on Raniwala's office network — run outbound traffic through a web-filtering/proxy appliance (the same class of system as Fortinet, Cisco Umbrella, Sophos, Zscaler, etc.) that automatically **categorizes every domain it sees** based on what content that domain actually serves, and then applies the company's policy per category (allow known-safe categories, block or challenge anything it can't confidently classify). A domain that returns an empty, content-less gateway response at its root — exactly what `api.crewcore.in` was doing — gets classified as “**uncategorized**”, and

on a network configured to block uncategorized domains by default (a common, reasonable default security posture for a corporate network), the *entire domain* gets blocked outright at the network level. Not one endpoint, not one request type — the whole domain, because the filter's decision is made per-domain, not per-request.

Because literally every backend call CrewCore makes goes through `api.crewcore.in`, once Raniwala's office network's web filter blocked that domain, **every single backend request from inside that office silently failed** — not because of anything wrong with CrewCore's code, not because of anything wrong with Supabase, and not because of anything wrong with the ESSL integration covered earlier in this document. It failed purely because Raniwala's own office network infrastructure had automatically decided `api.crewcore.in` was an unrecognized, “uncategorized” domain and was refusing to let any traffic to it leave the building at all. The static `crewcore.in` frontend kept loading fine throughout, because that's a different domain (Vercel-hosted, well-established, already categorized) — which is exactly why the symptom looked like “the page loads but nothing inside it works,” rather than a total site-down outage.

How It Was Fixed

The fix was to stop letting a plain request to `api.crewcore.in/` ever reach Supabase's bare gateway in the first place, by putting something in front of it that serves **real, readable content** at the root path — enough for any web-filtering system to correctly categorize the domain as a legitimate business API/service rather than flagging it “uncategorized,” while leaving every actual API call completely untouched.

This was implemented as a **Cloudflare Worker** (`cloudflare/api-crewcore-worker.js`), deployed on a Cloudflare Worker Route matching `api.crewcore.in/*`, sitting directly in front of the domain:

```
const ORIGIN = 'https://yxueywgqrfgynqknsqs.supabase.co';

export default {
  async fetch(request) {
    const url = new URL(request.url);

    // Only a plain GET to the root path gets the landing page.
    if (url.pathname === '/' && request.method === 'GET') {
      return new Response(LANDING_HTML, {
        status: 200,
        headers: { 'content-type': 'text/html; charset=utf-8' },
      });
    }

    // Every other path (auth, rest, functions, storage, realtime,
    // including websocket upgrades) passes straight through to
    // Supabase, completely unmodified.
    const originRequest = new Request(ORIGIN + url.pathname + url.search, request);
    return fetch(originRequest);
  },
};
```

What the Worker actually does, in plain terms:

- **If the request is a plain GET /** (exactly the shape of request a web-filter's categorization crawler makes) — it responds with a real, simple HTML page: a proper `<title>`, a `<meta name="description">` explaining that this is “CrewCore API backend service,” a robots meta tag allowing indexing, and a plain-language description with a link back to `api.crewcore.in`. This gives any filtering/categorization system genuine content to read and classify — turning `api.crewcore.in` from “uncategorized” into a normal, recognizable business API domain — which is enough for corporate web filters to stop auto-blocking it.
- **Every other request** — meaning every real thing the CrewCore app actually does: `/auth/...` (login/session), `/rest/...` (database reads/writes), `/functions/v1/...` (Edge Functions), `/storage/...` (file uploads), and the realtime/websocket connection — is forwarded straight through to the real Supabase origin (`yxueywgrqrfgynqknsqs.supabase.co`) completely unchanged, with the same path, query string, method, headers, and body. The Worker does not alter, delay, or add any overhead to actual API traffic in any way — it only intercepts the one specific request shape (GET /) that was causing the categorization problem.

Once this Worker was deployed in front of `api.crewcore.in`, the domain started returning real content at its root, corporate web filters (including Raniwala's) re-categorized it correctly, and it stopped being blocked — while every actual API call the app makes continued working exactly as it always had, since those paths were never touched by the Worker at all.

Current Status

Resolved. CrewCore now works normally from inside Raniwala's office network, the same as from anywhere else. Because the fix lives at the DNS/edge layer (a Cloudflare Worker Route in front of `api.crewcore.in`) rather than in CrewCore's own frontend or backend code, it protects every tenant using the app from this same class of problem going forward, not just Raniwala — any other office/ISP/network running a similar category-based web filter would have hit the identical “uncategorized domain” block on `api.crewcore.in` sooner or later; Raniwala's office was simply the first place it actually surfaced.